DATA MANIPULATION

KHOA CÔNG NGHỆ THÔNG TIN
TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN

Computer Science: An Overview by J. Glenn Brookshear and Dennis Brylow publisher Pearson

Data Manipulation

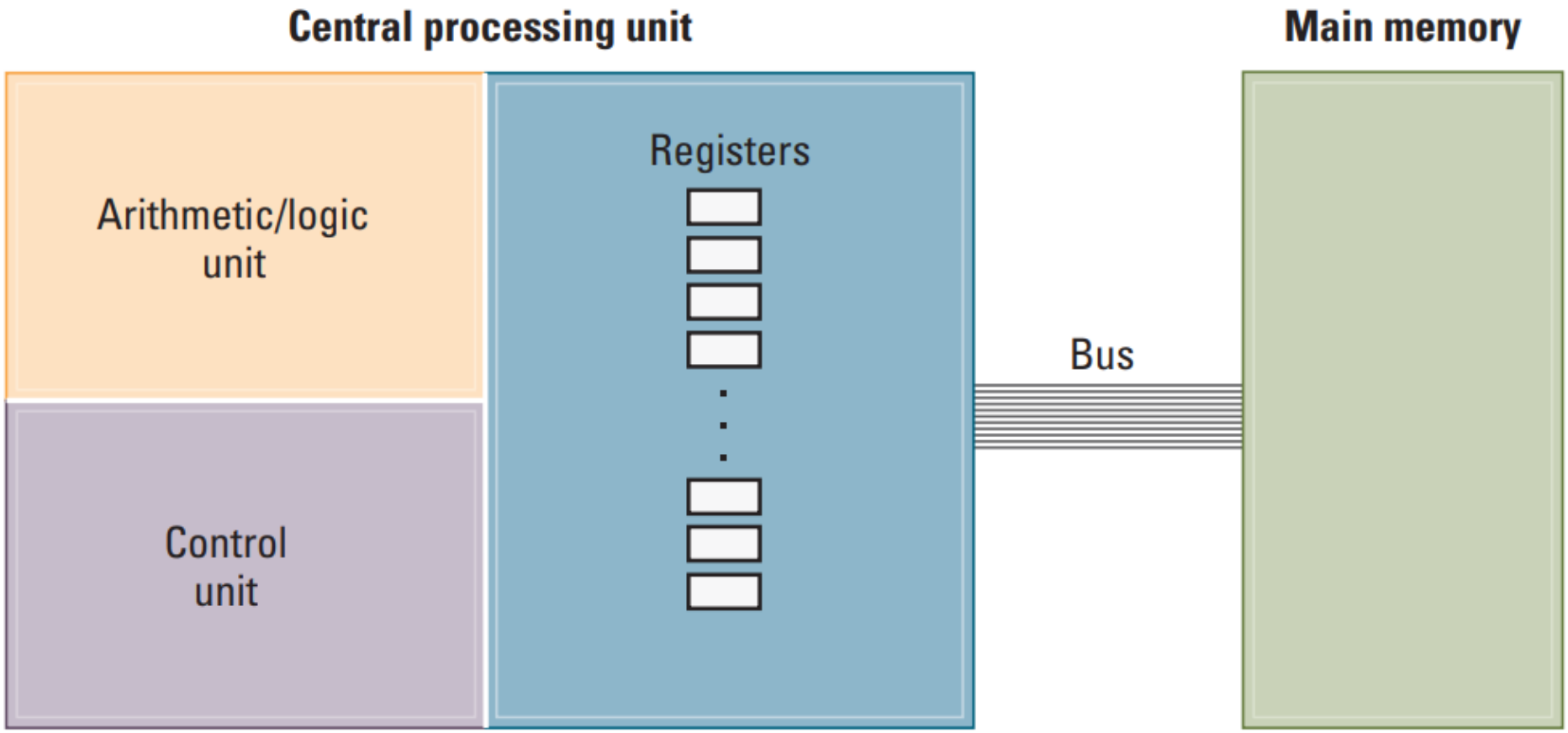
- Computer Architecture
- Machine Language
- Program Execution
- Arithmetic/Logic Instructions
- Communicating with Other Devices
- Other Architectures

COMPUTER ARCHITECTURE

Computer Architecture

- Central Processing Unit (CPU) or processor
  - Arithmetic/Logic unit versus Control unit
  - Registers
    - General purpose
    - Special purpose
- Bus
- Motherboard

CPU and main memory connected via a bus



Motherboards

- The motherboard
  - is the main printed circuit board.
  - contains the buses, or electrical pathways found in a computer. Buses allow data to travel among the various components.
  - accommodates CPU, RAM, expansion slots, heat sink/fan assembly, BIOS chip, chip set, sockets, internal and external connectors, various ports, and the embedded wires that interconnect the motherboard components.

Central Processing Unit

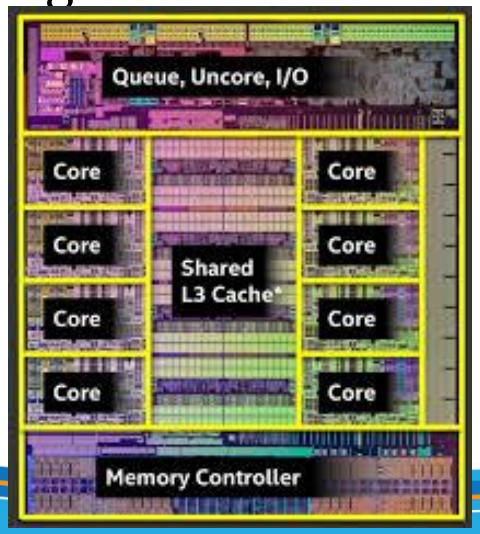
- The Central Processing Unit (CPU) is known as the brain of the computer. It is also referred to as the processor.
- The CPU executes a program, which is a sequence of stored instructions.

Central Processing Unit

- Some CPUs incorporate **hyperthreading** or **hypertransport** to enhance the performance of the CPU.
- The amount of data that a CPU can process at one time depends on the size of the processor data bus.
- Speed of the CPU is measured in **cycles per second** - megahertz (MHz) or gigahertz (GHz).
- **Overclocking** is a technique used to make a processor work at a faster speed than its original specification.

Central Processing Unit

- The latest processor technology has resulted in CPU manufacturers finding ways to incorporate more than one CPU core onto a single chip.
  - Dual Core CPU - Two cores inside a single CPU
  - Triple Core CPU - Three cores inside a single CPU
  - Quad Core CPU - Four cores inside a single CPU
  - Hexa-Core CPU - Six cores inside a single CPU
  - Octa-Core CPU - Eight cores inside a single CPU



The Arithmetic/Logic Unit

- Subsystem that performs addition, subtraction, and comparison for equality
- Components
  - Registers, interconnections between components, and the ALU circuitry
- Register
  - Storage cell that holds the operands of an arithmetic operation and holds its result
- Bus
  - Path for electrical signals

The Arithmetic/Logic Unit

- Registers are similar to RAM with following minor differences
  - They do not have a numeric memory address but are accessed by a special register designator such as A, X or R0
  - They can be accessed much more quickly than regular memory cells
  - They are not used for general purpose storage but for specific purposes such as holding the operands for an upcoming arithmetic computations.
- A typical ALU has 16, 32 or 64 registers.

The Control Unit

- Control unit
  - Tasks: fetch, decode, and execute

MACHINE LANGUAGE

Stored Program Concept

- A program can be encoded as bit patterns and stored in main memory.
- From there, the CPU can then extract the instructions and execute them.
- In turn, the program to be executed can be altered easily.

Terminology

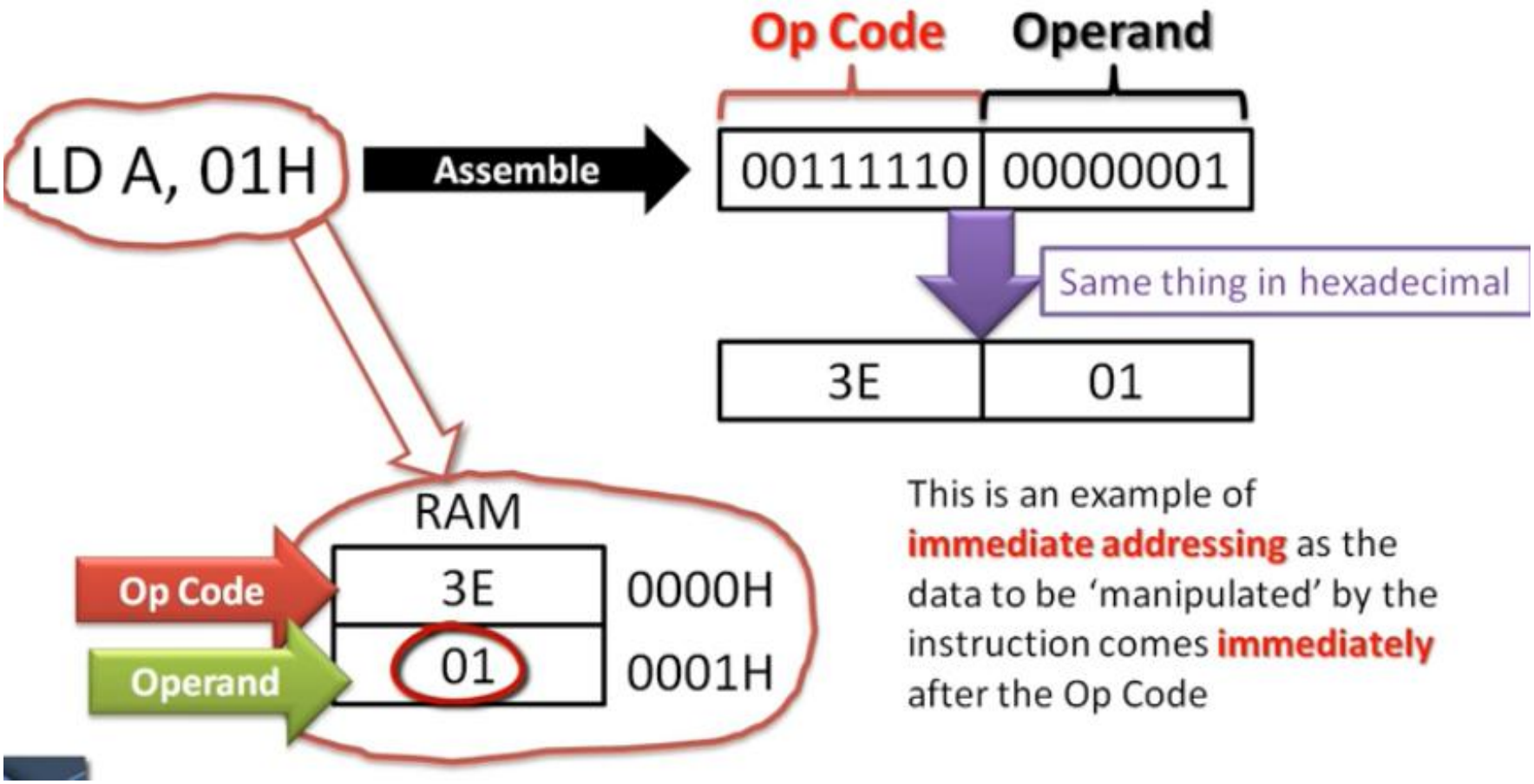
- Machine instruction: An instruction (or command) encoded as a bit pattern recognizable by the CPU

- Machine language: The set of all instructions recognized by a machine

Machine Language

ADD contents of 2 registers, store result in third.	1010000100 RR RR RR ex: R0 = R1 + R2 1010000100 00 01 10
SUBTRACT contents of 2 registers, store result into third	1010001000 RR RR RR ex: R0 = R1 – R2 1010001000 00 01 10
Halt the program	1111111111111111

Terminology



This is an example of **immediate addressing** as the data to be 'manipulated' by the instruction comes **immediately** after the Op Code

Machine Language Philosophies

- Reduced Instruction Set Computing (RISC)
  - Few, simple, efficient, and fast instructions
  - Examples: PowerPC from Apple/IBM/Motorola and ARM

- Complex Instruction Set Computing (CISC)
  - Many, convenient, and powerful instructions
  - Example: Intel

RISC vs CISC

- CISC (x86) - Single Complex Instruction

```
MULADD eax, [memory1], [memory2], [memory3]
```

- RISC (ARM) - Multiple Simple Instructions

```
LDR  r1, [memory1]      ; Load first value into register
LDR  r2, [memory2]      ; Load second value
LDR  r3, [memory3]      ; Load third value
MUL  r4, r1, r2         ; Multiply first two
ADD  r0, r4, r3         ; Add third value
```

- Five instructions, but each is simple and fast

RISC advantages

Even though RISC uses more instructions:

- Each RISC instruction completes in one clock cycle (typically)
- CISC complex instructions take multiple clock cycles
- RISC's uniform instructions allow better pipelining
- Modern CPUs can execute multiple RISC instructions in parallel

Machine Instruction Types

- **Data Transfer:** copy data from one location to another
- **Arithmetic/Logic:** use existing bit patterns to compute a new bit patterns
- **Control:** direct the execution of the program

Adding values stored in memory

Step 1. Get one of the values to be added from memory and place it in a register.

Step 2. Get the other value to be added from memory and place it in another register.

Step 3. Activate the addition circuitry with the registers used in Steps 1 and 2 as inputs and another register designated to hold the result.

Step 4. Store the result in memory.

Step 5. Stop.

Dividing values stored in memory

Step 1. LOAD a register with a value from memory.

Step 2. LOAD another register with another value from memory.

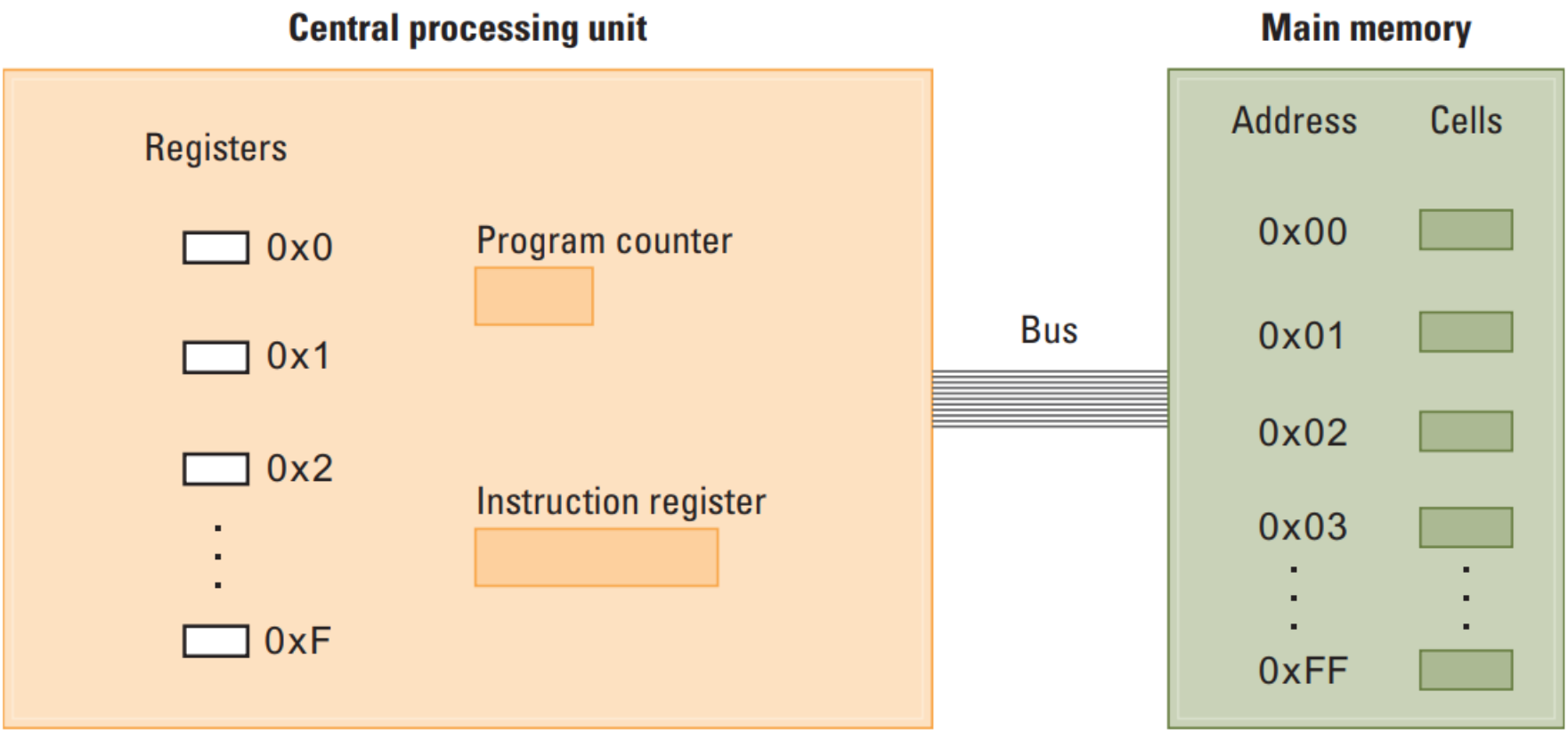
Step 3. If this second value is zero, JUMP to Step 6.

Step 4. Divide the contents of the first register by the second register and leave the result in a third register.

Step 5. STORE the contents of the third register in memory.

Step 6. STOP.

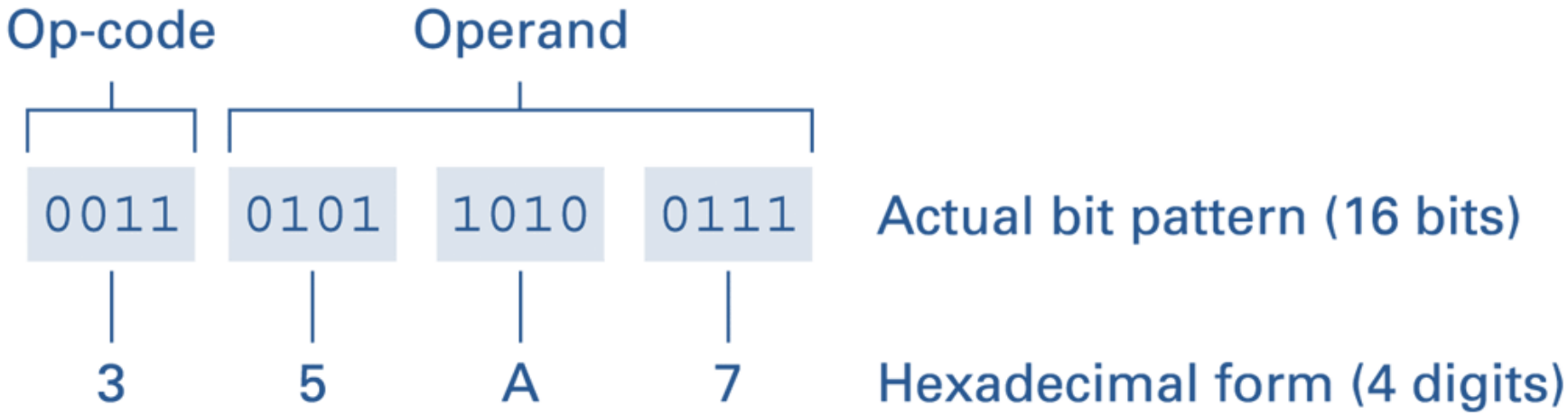
An architecture of the machine



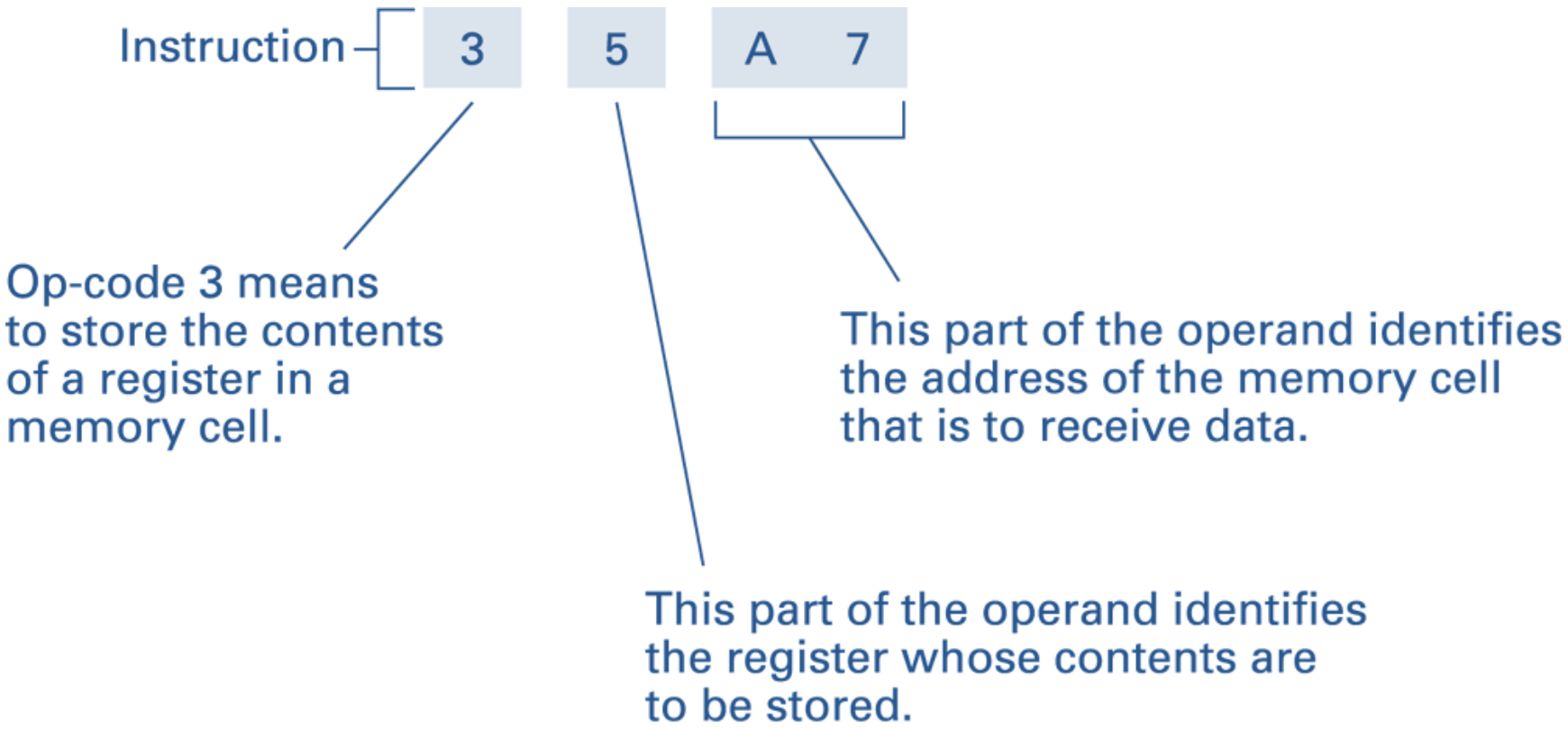
Parts of a Machine Instruction

- **Op-code**: Specifies which operation to execute
- **Operand**: Gives more detailed information about the operation
  - Interpretation of operand varies depending on op-code

The composition of an instruction



Decoding the instruction 35A7



A Simple Machine Language

Op-code	Operand	Description
1	RXY	LOAD reg. R from cell XY.
2	RXY	LOAD reg. R with XY.
3	RXY	STORE reg. R at XY.
4	0RS	MOVE R to S.
5	RST	ADD S and T into R. (2's comp.)
6	RST	ADD S and T into R. (floating pt.)

A Simple Machine Language

Op-code	Operand	Description
7	RST	OR S and T into R.
8	RST	AND S and T into R.
9	RST	XOR S and T into R.
A	R0X	ROTATE reg. R X times.
B	RXY	JUMP to XY if R = reg. 0.
C	0	HALT.

A Simple Machine Language

- **1**4A3: Load
  - The contents of the **memory cell** located at address **A3** to be placed in register 4.
- **2**0A3: Load
  - The value **A3** to be placed in register 0.
- **3**5B1: Store
  - The contents of register 5 to be placed in the memory cell whose address is **B1**.
- **4**0A4: Move
  - The contents of register **A** to be copied into register **4**.

A Simple Machine Language

- 5726: Add
  - The binary values in registers 2 and 6 to be added and the sum placed in register 7.
- 634E: Add
  - The values in registers 4 and E to be added as f**loating-point** values and the result to be placed in register 3.

A Simple Machine Language

- 7CB4: Or
  - The result of ORing the contents of registers B and 4 to be placed in register C.
- 8045: And
  - The result of ANDing the contents of registers 4 and 5 to be placed in register 0.
- 95F3: Xor
  - The result of XORing the contents of registers F and 3 to be placed in register 5.
- A403: Rotate
  - The contents of registers 4 to be rotated 3 bits to the right in a circular fashion.

A Simple Machine Language

- B43C:
  - Compare the contents of register **4** with the contents of register 0.
  - If equal, the pattern **3C** would be placed in the program counter so that the next execution executed would be the one located at that memory address.
  - Otherwise, nothing would be done.
- C000:
  - Stop program execution.

An encoded version of the instructions

Encoded instructions	Translation
156C	Load register 5 with the bit pattern found in the memory cell at address 6C.
166D	Load register 6 with the bit pattern found in the memory cell at address 6D.
5056	Add the contents of register 5 and 6 as though they were two's complement representation and leave the result in register 0.
306E	Store the contents of register 0 in the memory cell at address 6E.
C000	Halt.

Quiz

- The following are instructions written in the above simple machine language. Rewrite in English.
  - 368A:
  - BADE
  - 803C
  - 40F4

Quiz

- What is the difference between the instructions **15AB** and **25AB** in the simple machine language?

15AB -> nạp dữ liệu từ địa chỉ ô nhớ AB vào thanh ghi 5

25AB -> nạp giá trị AB vào thanh ghi 5

Quiz

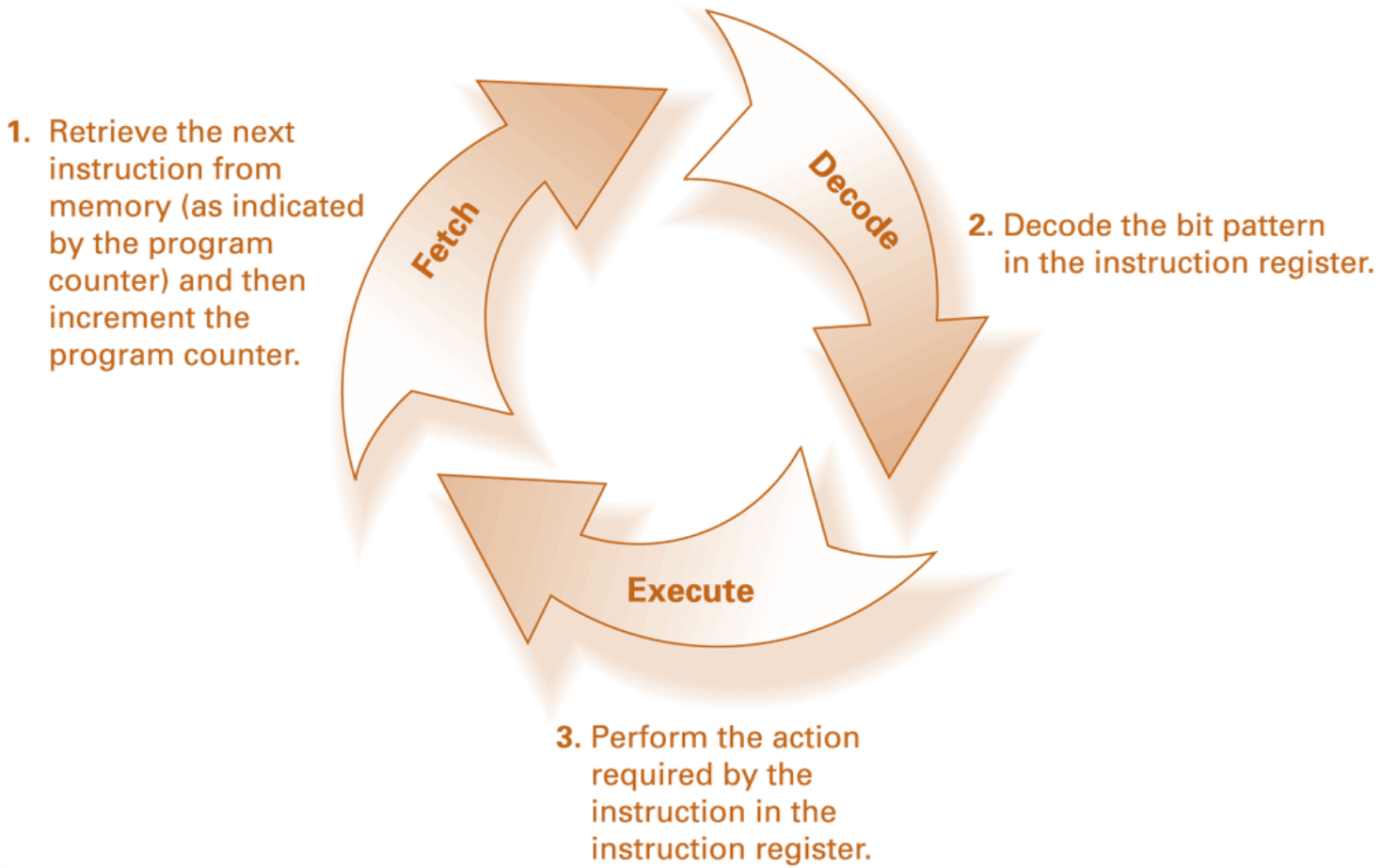
- Here are some instructions in English. Translate each of them into the simple machine language.
  - LOAD register number 3 with the hexadecimal 56.
  - ROTATE register number 5 three bits to the right.
  - AND the contents of register A with the contents of register 5 and leave the result in register 0.

PROGRAM EXECUTION

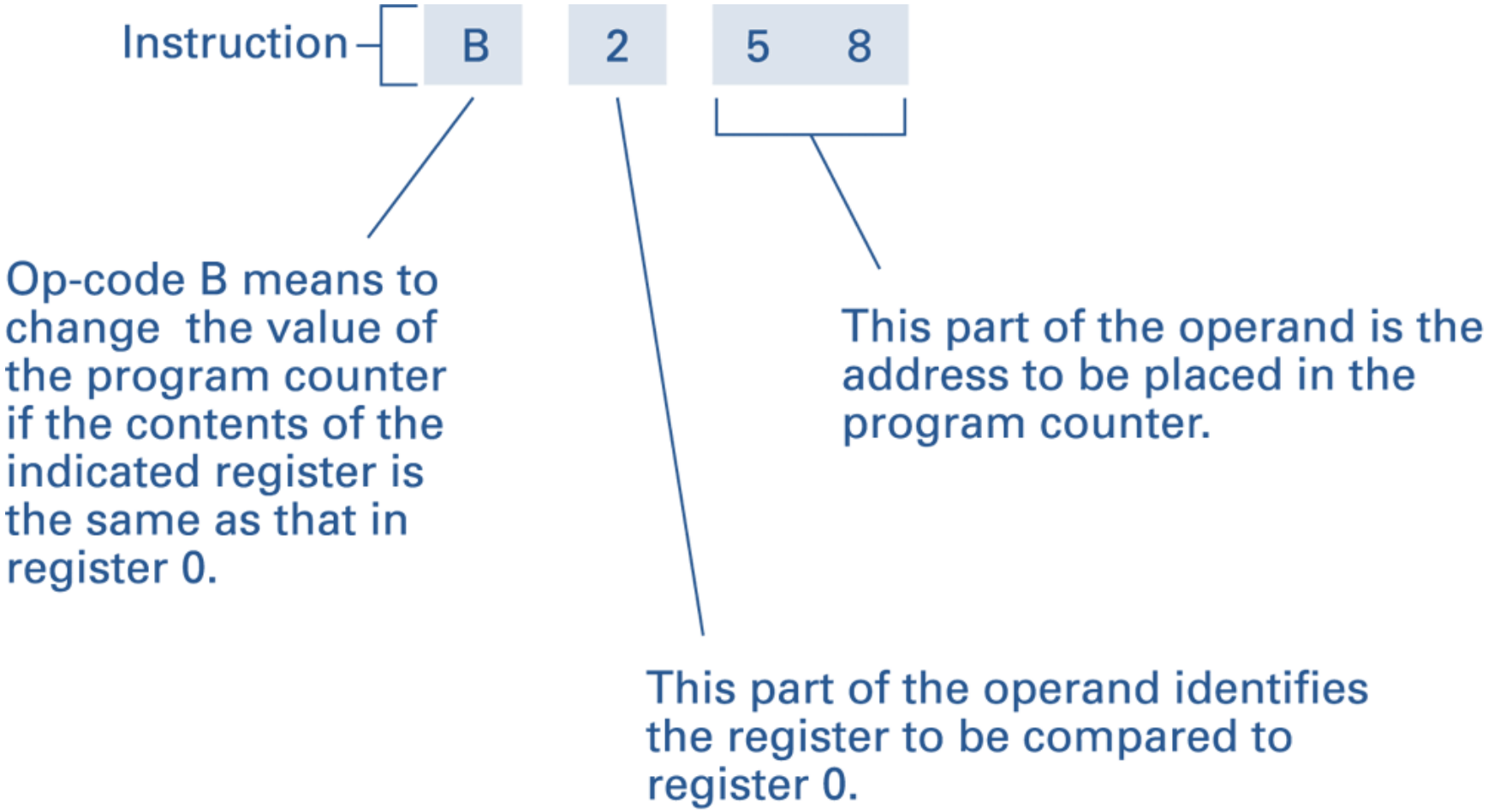
Program Execution

- Controlled by two special-purpose registers
  - Program counter: address of next instruction
  - Instruction register: current instruction
- Machine Cycle
  - Fetch
  - Decode
  - Execute

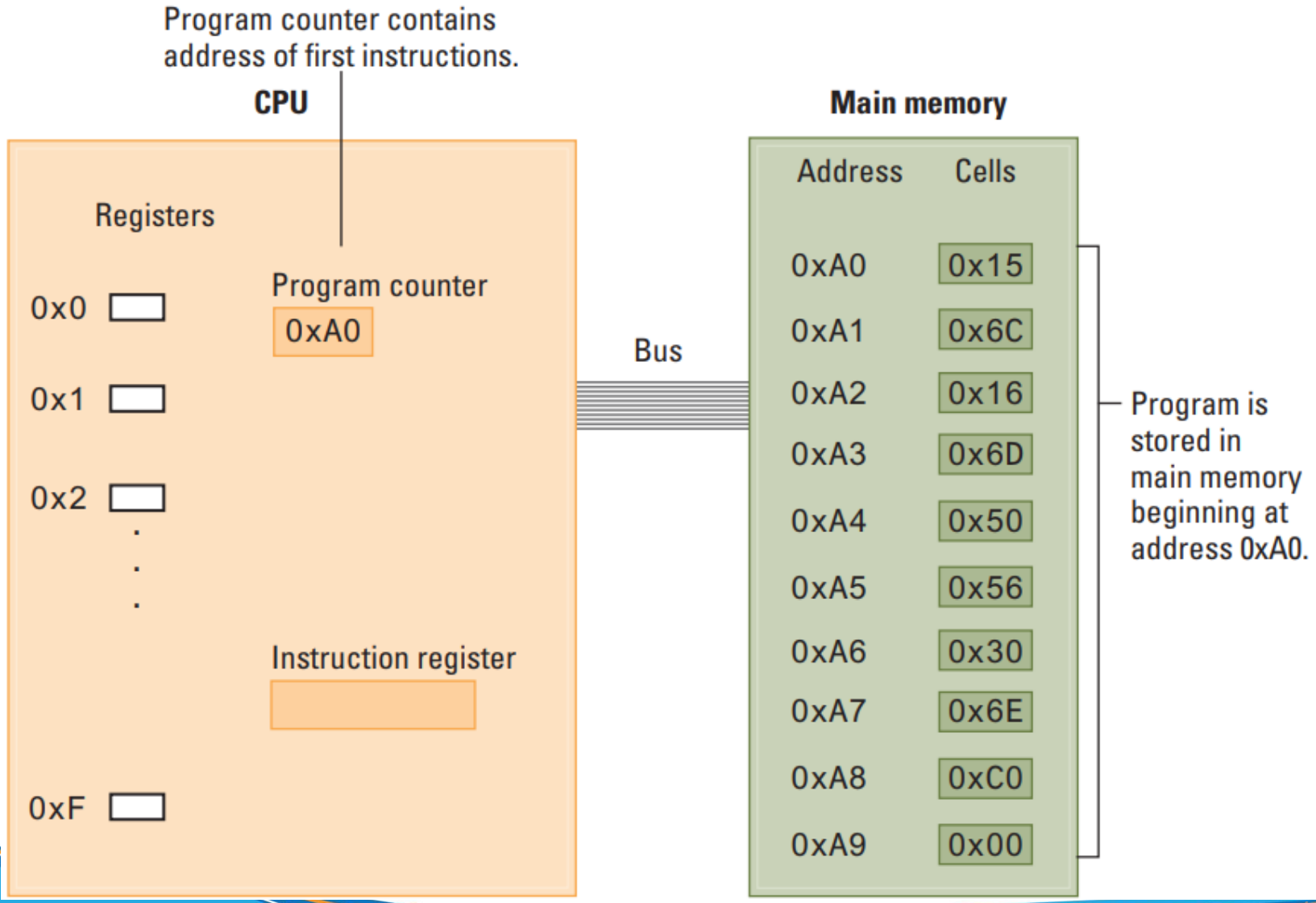
The machine cycle



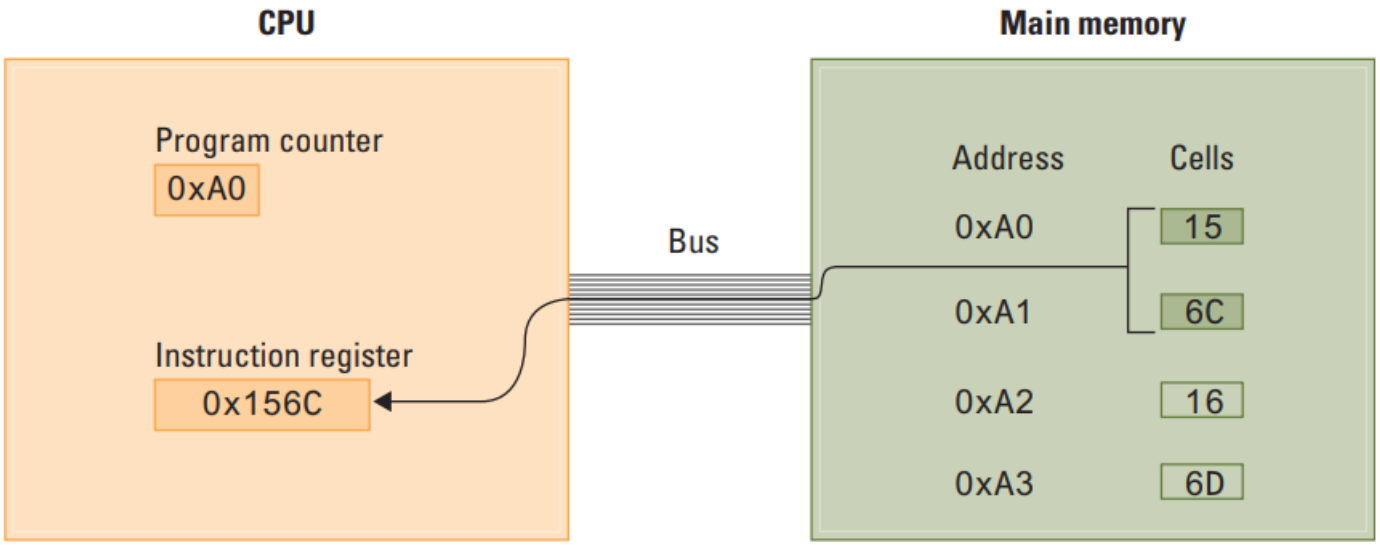
Decoding the instruction B258



A program stored in main memory

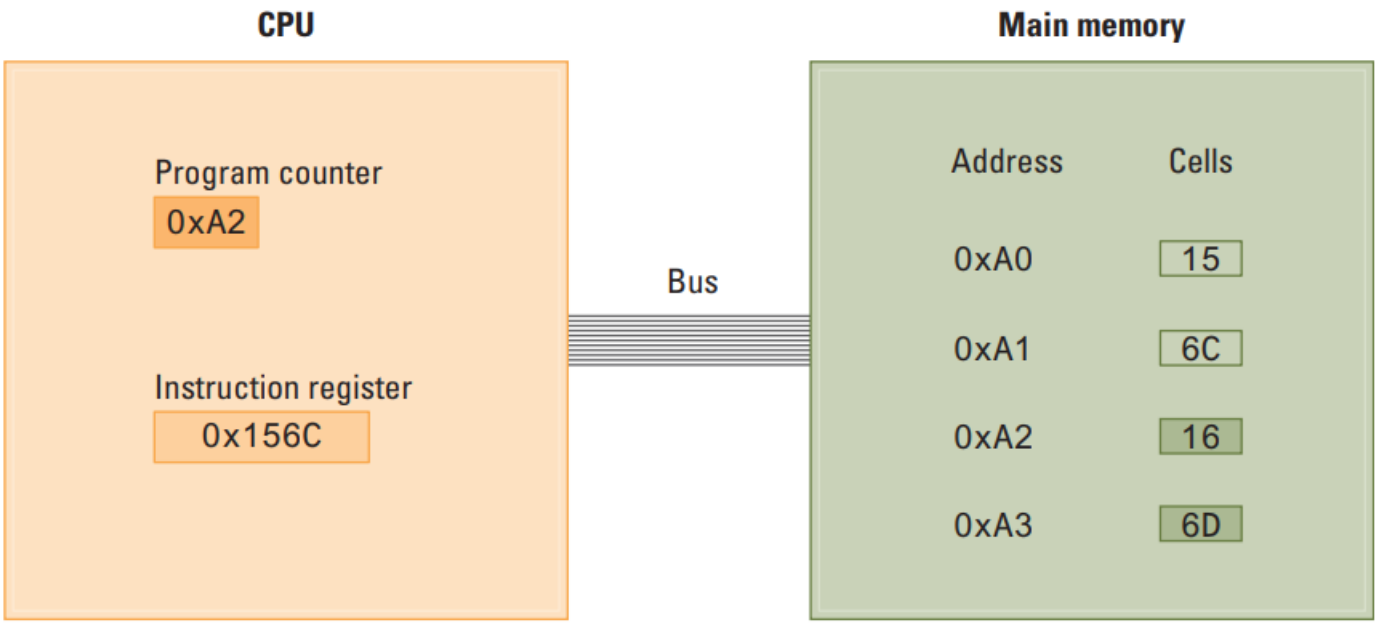


The fetch step of the machine cycle



a. At the beginning of the fetch step, the instruction starting at address 0xA0 is retrieved from memory and placed in the instruction register.

The fetch step of the machine cycle



ARITHMETIC/LOGIC INSTRUCTIONS

How ALU work

- **Control Unit** sends operation code (opcode)
  Example: "ADD" instruction
- **Operands** loaded into input registers
  Register A = 5, Register B = 3
- **ALU** performs operation using logic gates
  Internal circuits add the numbers
- Result stored in output **register**
  Result = 8
- **Status flags** updated
  Zero flag = 0, Carry flag = 0

Arithmetic/Logic Operations

- Logic: AND, OR, XOR
  - Masking
- Rotate and Shift:
  - circular shift (rotation)
  - logical shift
  - arithmetic shift
- Arithmetic: add, subtract, multiply, divide
  - Precise action depends on how the values are encoded (two's complement versus floating-point).

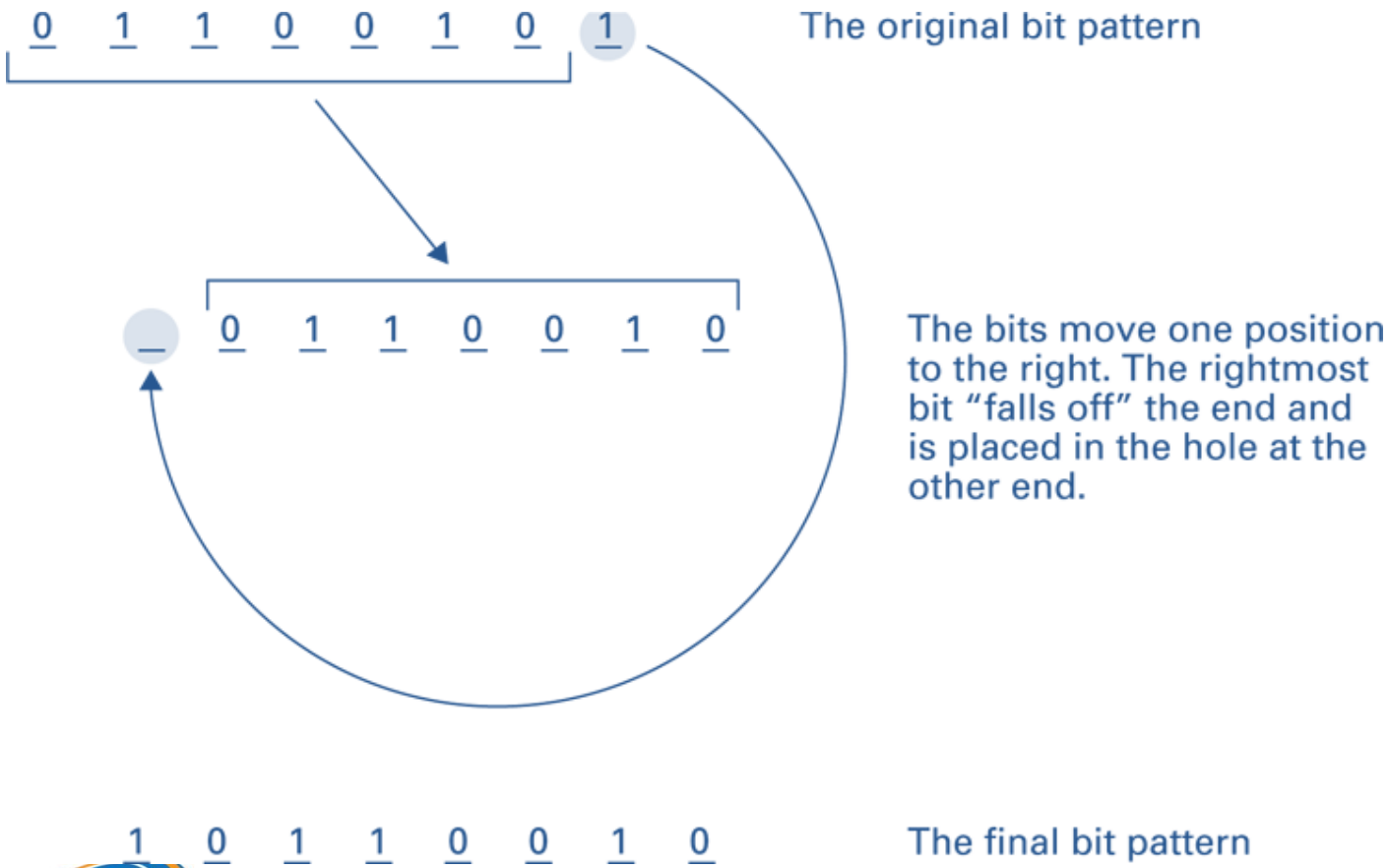
Masking

```
     00001111
AND  10101010
     00001010
```

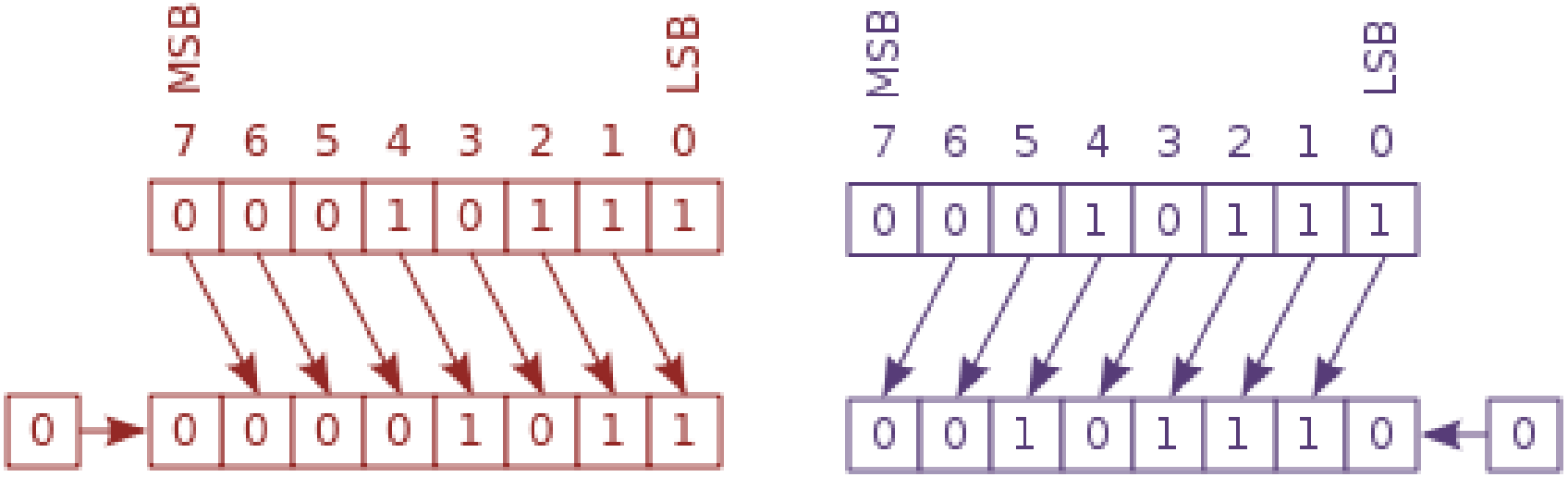
```
     11110000
OR   10101010
     11111010
```

```
     11111111
XOR  10101010
     01010101
```

Circular shift



Logical shift



QUIZ

Quiz

- What logical operation together with what mask can you use to change ASCII codes of lower characters to uppercase? What about uppercase to lowercase?

Symbol	ASCII	Hex	Symbol	ASCII	Hex
>	00111110	3E	^	01011110	5E
?	00111111	3F	_	01011111	5F
@	01000000	40	`	01100000	60
A	01000001	41	a	01100001	61
B	01000010	42	b	01100010	62
C	01000011	43	c	01100011	63
D	01000100	44	d	01100100	64
E	01000101	45	e	01100101	65
F	01000110	46	f	01100110	66
G	01000111	47	g	01100111	67
H	01001000	48	h	01101000	68
I	01001001	49	i	01101001	69
J	01001010	4A	j	01101010	6A
K	01001011	4B	k	01101011	6B
L	01001100	4C	l	01101100	6C

Quiz

- Identify both the mask and the logical operation needed to accomplish each of the following objectives:
- Put 1s in the upper 4 bits of an 8-bit pattern without disturbing the other bits.
- Complement the most significant bit of an 8-bit pattern without changing the other bits.
- Complement a pattern of 8 bits.
- Put a 0 in the least significant bit of an 8-bit pattern without disturbing the other bits.

Quiz

- Identify both the mask and the logical operation needed to accomplish each of the following objectives:
- Put 1s in the upper 4 bits of an 8-bit pattern without disturbing the other bits.

  OR 1111 0000
- Complement the most significant bit of an 8-bit pattern without changing the other bits.

  XOR 1000 0000
- Complement a pattern of 8 bits.

  XOR 1111 1111
- Put a 0 in the least significant bit of an 8-bit pattern without disturbing the other bits.

  AND 1111 1110

Quiz

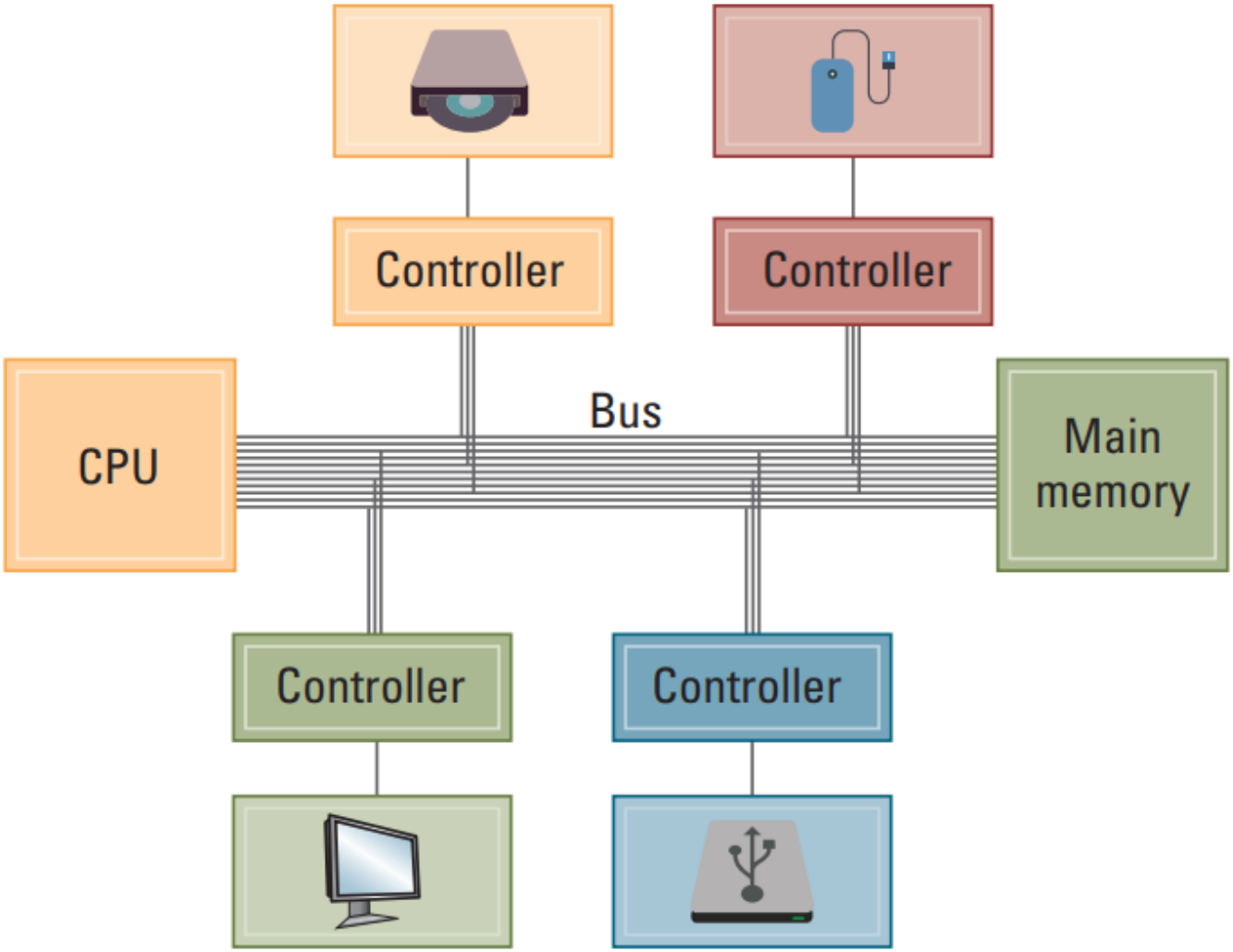
- Identify both the mask and the logical operation needed to accomplish each of the following objectives:
- Put 1s in all but the most significant bit of an 8-bit pattern without disturbing the most significant bit.
- Filter out all of the green color component from an RGB bitmap image pixel in which the middle 8 bits of a 24-bit pattern store the green information.
- Invert all of the bits in a 24-bit RGB bitmap
- Set all the bits in a 24-bit RGB bitmap pixel to 1, indicating the color “white”.

COMMUNICATING WITH OTHER DEVICES

Communicating with Other Devices

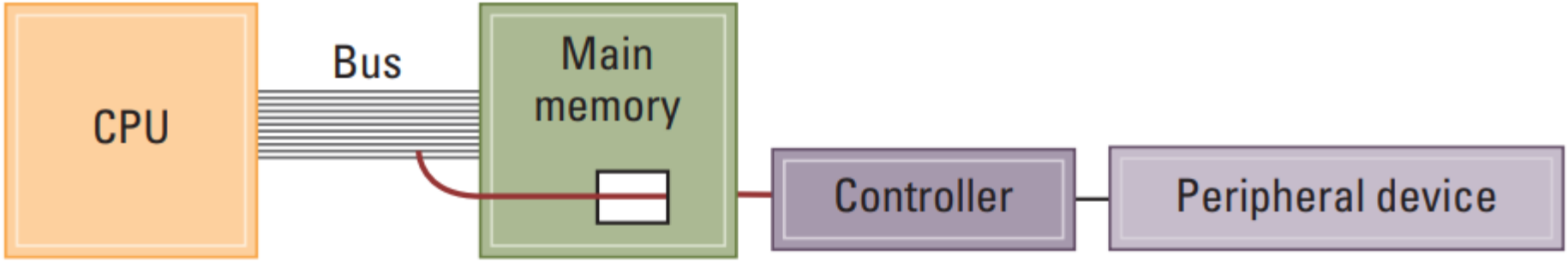
- **Controller**: An intermediary apparatus that handles communication between the computer and a device
  - Specialized controllers for each type of device
  - General purpose controllers (USB and FireWire)
- **Port**: The point at which a device connects to a computer
- **Memory-mapped I/O**: CPU communicates with peripheral devices as though they were memory cells

Controllers attached to a machine's bus



A conceptual representation of memory-mapped I/O

- CPU communicate with the controllers attached to the bus in the same manner that it communicates with main memory.
- STORE: send a bit pattern to a controller
- LOAD: receive a bit pattern from a controller



Communicating with Other Devices

- Direct memory access (DMA): Main memory access by a controller over the bus
- Ex: copy a large file from your hard disk to RAM
- **Without DMA**:
  1. CPU tells disk controller: "Read block 100“
  2. Disk controller reads data from disk into its buffer
  3. Disk controller signals CPU: "Data ready“
  4. CPU reads one byte from disk controller
  5. CPU writes that byte to memory
  6. Repeat steps 4-5 for EVERY BYTE (thousands or millions of times!)
  7. Finally done

Communicating with Other Devices

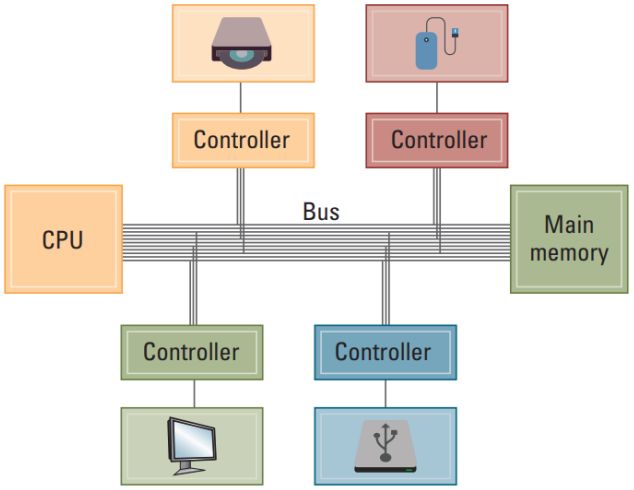
- Direct memory access (DMA): Main memory access by a controller over the bus
- Ex: copy a large file from your hard disk to RAM
- **Without DMA**:
  - The CPU must personally handle every single byte of data transfer
  - For a 1 MB file, the CPU does this loop 1,048,576 times!
  - The CPU can't do any other useful work during this time
  - This is incredibly wasteful—the CPU is just a glorified "data mover"

Communicating with Other Devices

- **With DMA**:
  - CPU tells DMA controller: "Copy 1 MB from disk controller to memory address 0x100000“
  - CPU goes off to do OTHER WORK (running calculations, etc.)
  - DMA controller handles the entire transfer:
    - Reads data from disk controller
    - Writes data directly to memory
    - Keeps track of how many bytes transferred
    - Increments memory addresses automatically
  - When complete, DMA controller interrupts CPU: "Transfer done!“
  - CPU: "Great!" and continues with next task

Communicating with Other Devices

- Von Neumann Bottleneck: Insufficient bus speed impedes performance
  - The use of DMA has the detrimental effect of complicating the communication taking place over a bus.
  - Bit patterns must move between the CPU and main memory, between the CPU and each controller, and between each controller and main memory.
  - Coordination of all this activity on the bus is a major design issue.



Communicating with Other Devices

- Handshaking: the computer and the peripheral device exchange information about the device's status and coordinate their activities
- Handshaking often involves a status word, a bit pattern that is generated by the peripheral device and sent to the controller.
- The status word is a bit map in which the bits reflect the conditions of the device
- E.g.: printer is out of paper, printer is ready for additional data, paper jam

Communicating with Other Devices

- Parallel Communication: Several communication paths transfer bits simultaneously.
- Serial Communication: Bits are transferred one after the other over a single communication path.

Serial Communication

Parallel Communication

Communicating with Other Devices

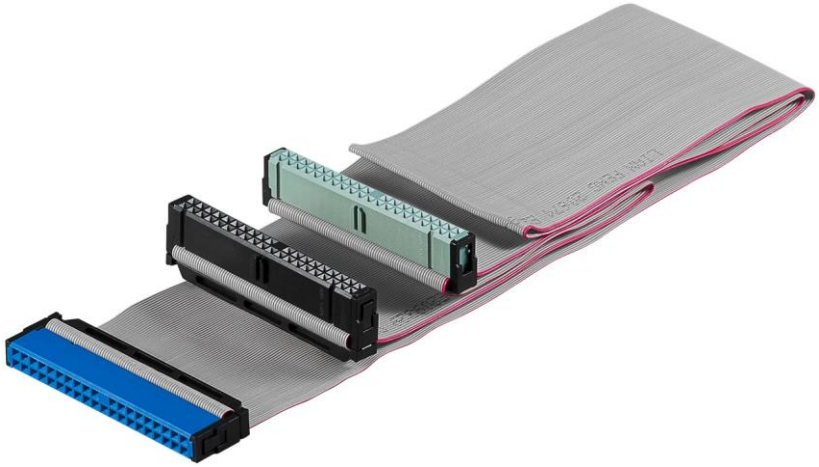
- Why Serial bus better nowdays:
  - Clock skew: signal arrives at different components at different times
  - Crosstalk: a signal transmitted on channel of a transmission system creates an undesired effect in another channel
  - Cheaper

Communicating with Other Devices

- Example of Parallel bus: very short distance
  - Internal buses
  - Legacy computer peripheral buses: ISA, ATA, SCSI, PCI
- Example of Seiral bus: mid to long range
  - Ethernet
  - HDMI, PCI Express, Serial ATA
  - Universal Serial Bus (USB)

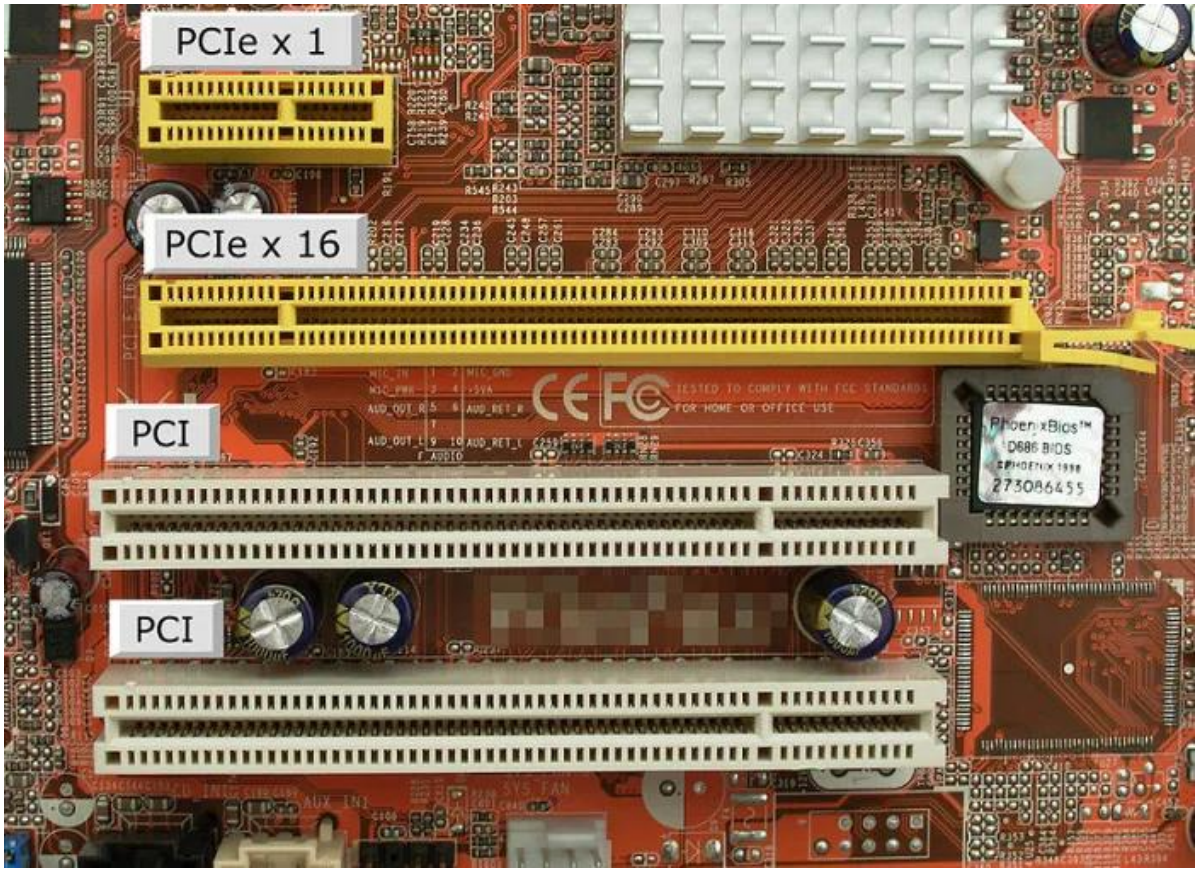
Communicating with Other Devices

- ATA vs SATA



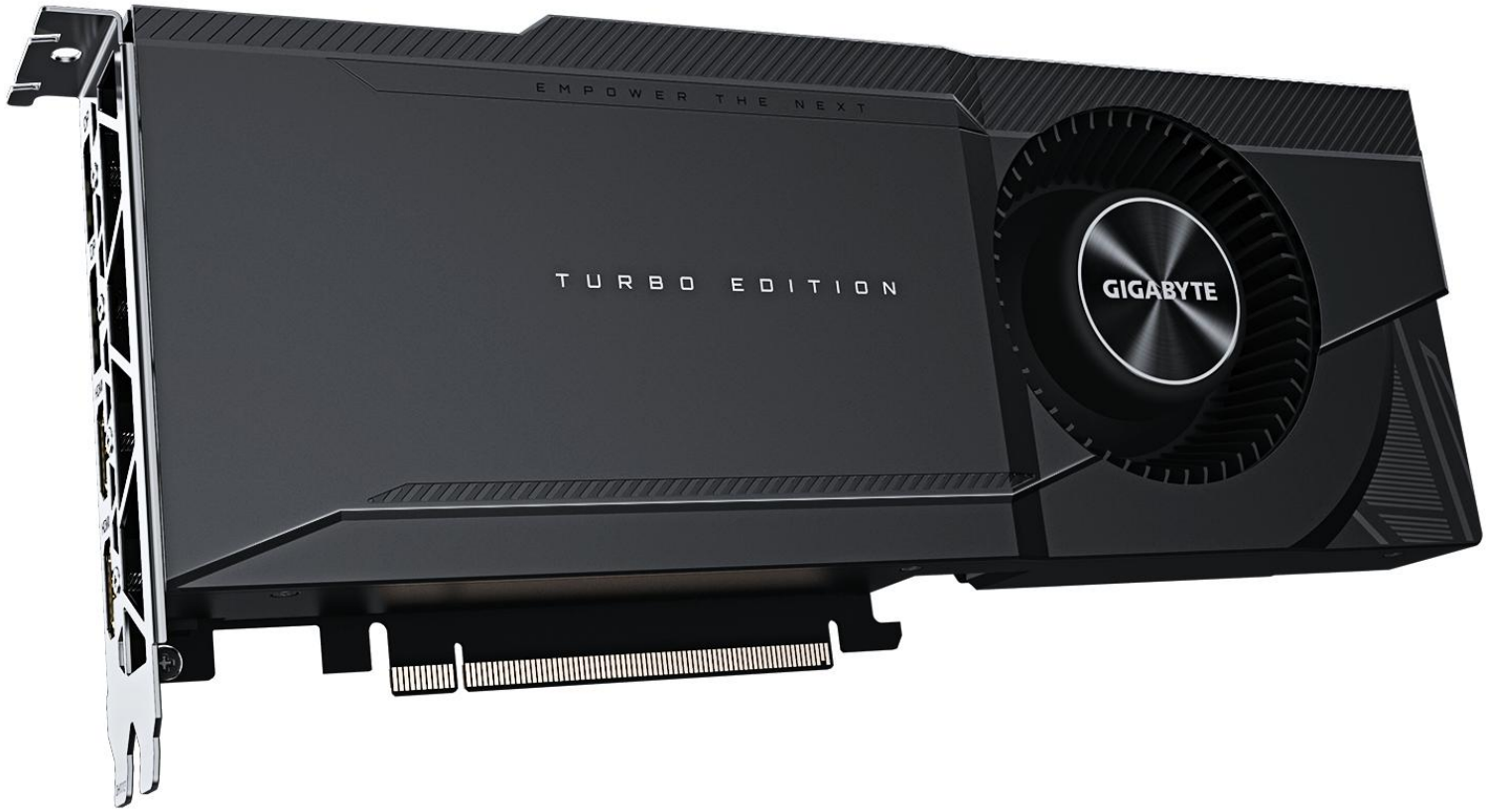
Communicating with Other Devices

- PCI vs PCI Express



Communicating with Other Devices

- PCI vs PCI Express



Parallel Communication Still Used

- RAM/Memory InterfacesInternal
- CPU Buses
- Address Buses

Data Communication Rates

- Measurement units
  - Bps: Bits per second
  - Kbps: Kilo-bps (1,000 bps)
  - Mbps: Mega-bps (1,000,000 bps)
  - Gbps: Giga-bps (1,000,000,000 bps)
- Bandwidth: Maximum available rate

Quiz

- Assume that the machine (using the simple machine language as described) uses memory-mapped I/O and that the address B5 is the location within the printer port to which data to be printed should be sent.

a. If register 7 contains the ASCII code for the letter A, what machine language instruction should be used to cause that letter to be printed at the printer?

b. If the machine executes a million instructions per second, how many times can this character be sent to the printer in one second?

Quiz

- Assume that the machine (using the simple machine language as described) uses memory-mapped I/O and that the address B5 is the location within the printer port to which data to be printed should be sent.

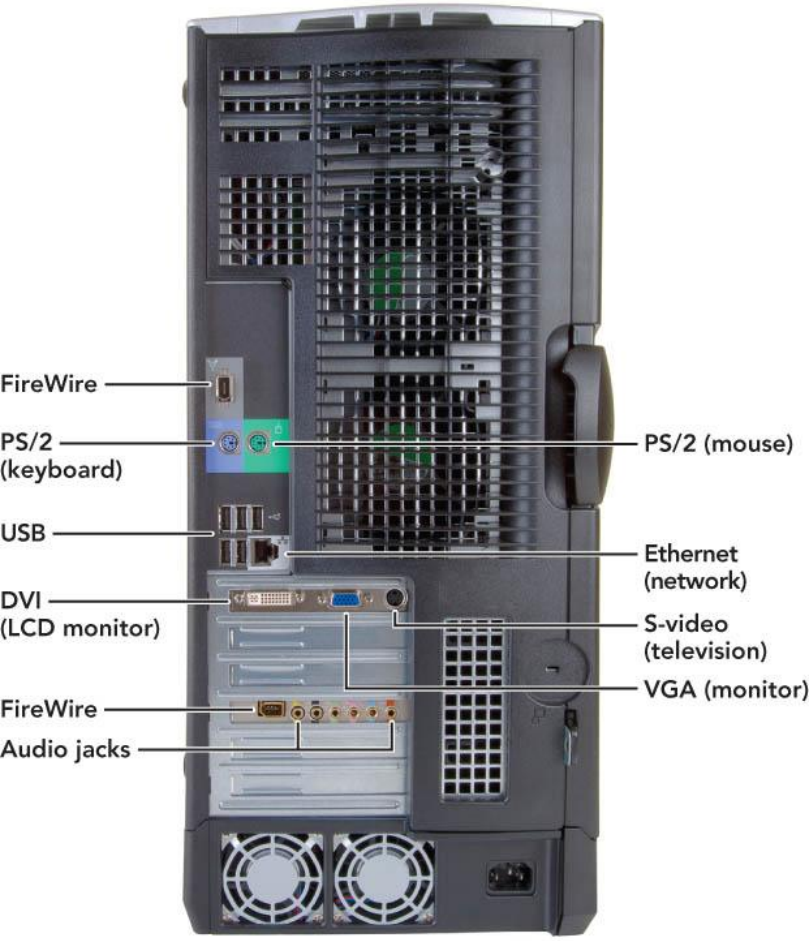
c. If the printer is capable of printing 60 traditional pages of text per minute, will it be able to keep up with the characters being sent to it in (b)?

Knowing that: typical page of text contains less than 4000 characters

Quiz

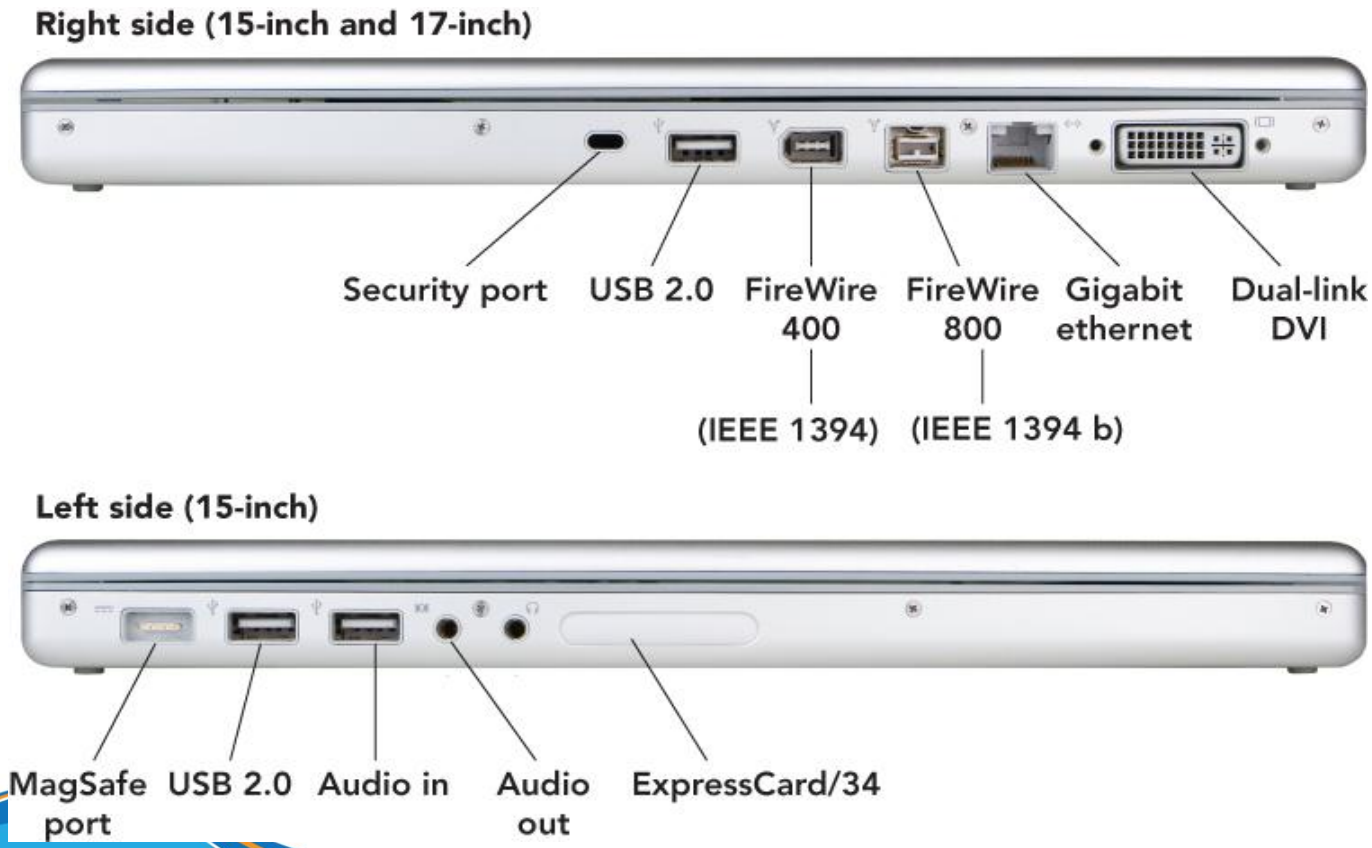
- Estimate how long it would take to transfer a 250-page novel (about 500,000 characters) encoded in 16-bit Unicode characters at a transfer rate of 54Mbps.

What's on the Outside of the Box?



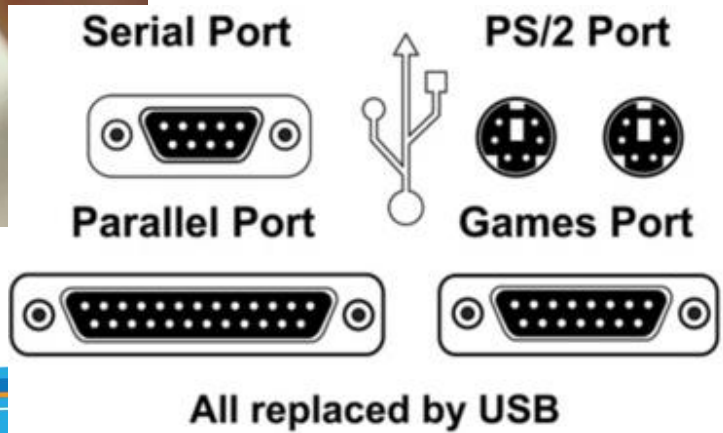
What's on the Outside of the Box?

- Connectors on a notebook may vary



Universal serial bus

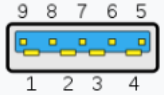
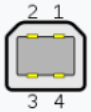
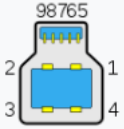
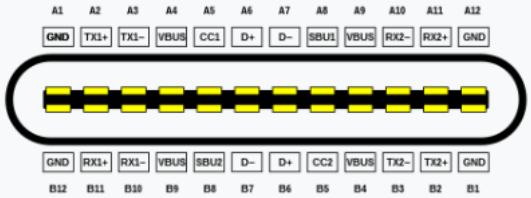
- USB (universal serial bus) ports
  - Connects up to 127 peripheral devices
  - USB 2.0 (high-speed USB)—fully compatible with USB 1.1 products, cables, and connectors
  - Designed to replace older parallel and serial ports
  - Connects a variety of devices to the computer, including:
    - Keyboards
    - Mice
    - Printers
    - Digital cameras



Universal serial bus

- USB 2.0
  - Uses an external bus
  - Supports data transfer rates of 480 Mbps between the computer and the peripheral device
  - Supports hot swapping—ability to connect and disconnect devices without shutting down the computer
  - Plug-and-play (PnP)—allows computers to automatically detect the device when you plug it in
- USB 3.0 (2008): 5Gbps
- USB 3.1 (2013): 10Gbps
- USB 3.2 (2017): 20Gbps
- USB4 (2019): 40Gbps
- USB4 2.0 (2022 – Sept.): 80Gbps
- USB hub
  - Device that plugs into existing USB port
  - Contains four or more additional ports

Universal serial bus

Standard	USB 1.0 1996	USB 1.1 1998	USB 2.0 2001	USB 2.0 Revised	USB 3.0 2008	USB 3.1 2013	USB 3.2 2017	USB4 2019
Maximum transfer rate	12 Mbps		480 Mbps		5 Gbps	10 Gbps	20 Gbps	40 Gbps
Type A connector	Type-A 1.0 - 1.1		Type-A 2.0		 Type-A SuperSpeed		Deprecated	
Type B connector	 Type-B				 Type-B SuperSpeed		Deprecated	
Type C connector	Backwards compatibility only							

Video connectors

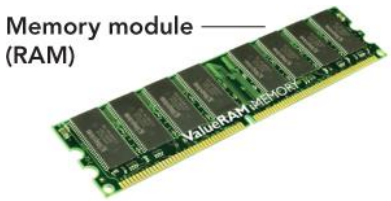
- VGA (video graphics array)
  - 15-pin male connector—works with standard monitor cables
  - Transmits analog video signals
  - Used for legacy technology cathode ray (CRT) monitors
- DVI (Digital visual Interface) port—lets LCD monitors use digital signals
- Onboard video—video circuitry built into the motherboard where the video connector is on the back of the system unit case

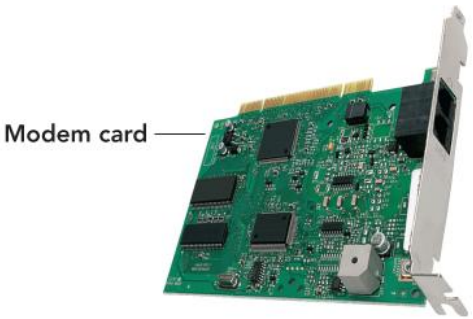
HDMI

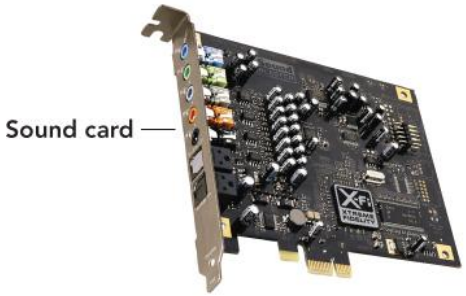
- High-Definition Multimedia Interface
  - A video/audio interface for transmitting uncompressed video data, and compressed/uncompressed audio data.
  - Digital replacement for analog video standards.
  - Designed: 2002 (7 companies).
  - HDMI 2.0: 18Gbps
  - HDMI 2.1: 48Gbps.
  - Newer version: 3D, Ethernet data connection, Consumer Electronics Control (CEC) extensions.

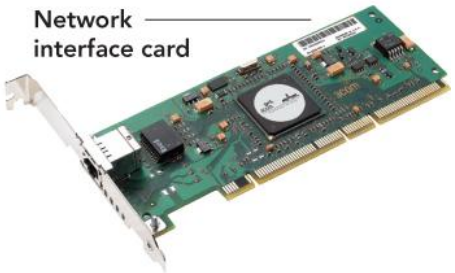
Additional connectors

- Telephone
- Network
- PC card slot
  - PC card
  - ExpressCard
- Sound card
- Game card
- TV/sound capture board







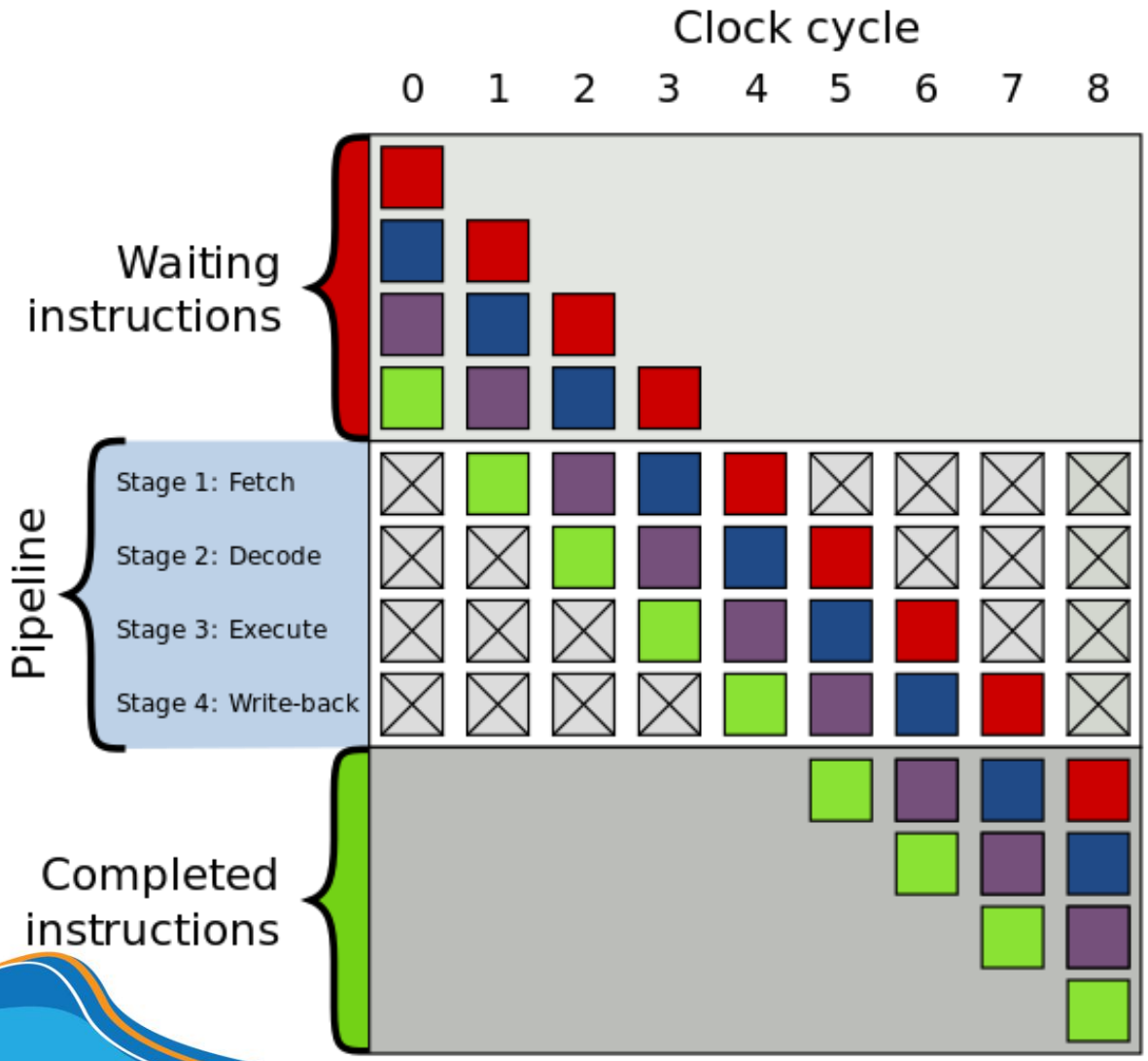


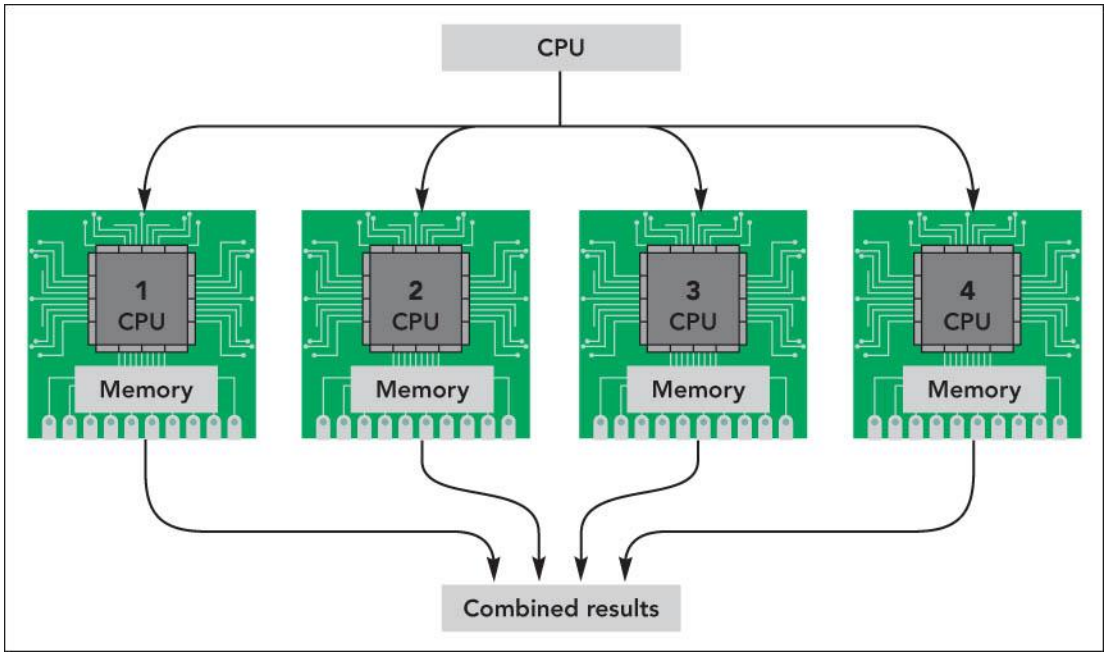
OTHER ARCHITECHTURES

Other Architectures

- Technologies to increase throughput:
  - Pipelining: Overlap steps of the machine cycle
  - Parallel Processing: Use multiple processors simultaneously
    - SISD: No parallel processing
    - MIMD: Different programs, different data
    - SIMD: Same program, different data

Pipelining


Clock cycle
0
1
2
3
4
5
6
7
8
Waiting instructions
Pipeline
Stage 1: Fetch
Stage 2: Decode
Stage 3: Execute
Stage 4: Write-back
Completed instructions

Parallel processing

- Method where more than one processor performs at the same time—faster processing